

INDEX

SL. NO	TOPICS	SIGN
01.	Write a program to create, Initialize and access a pointer variable	
02.	Write a program to calculate the length of the string using a pointer	
03.	Write a program to swap numbers using pointer	
04.	Write a program in c to print all permutations	
05.	Write a program to store n students information using structure	
06.	Write a program to implement push, pop and traverse operation on STACK	
07.	Write a program to convert infix notation to postfix notation	
08.	Write a program to convert infix notation to prefix notation	
09.	Write a program to convert prefix notation to postfix notation	
10.	Write a program to perform the operation insert, delete, Display on Queue.	
11.	Write a program to implement circular queue.	
12.	Write a program to implement Double ended queue.	
13.	Write a program to implement priority Queue.	
14.	Write a program to search an element using linear search.	

INDEX

[illegible]

Lab Program - 02

② C program to calculate length of the string using pointers

```
#include <stdio.h>
#include <conio.h>
int main()
{
    char s[10], *p;
    int i;
    printf("enter a string : \t");
    gets(s);
    p = s;
    while(*p != '\0')
    {
        i++;
        p++;
    }
    printf("length of string is %d", i);
    return 0;
}
```

Output

Enter a string 'Neha'
length of string is 4

Lab Program -01

① Write a Program to create, initialize and access a pointer variable

```
#include <stdio.h>
#include <conio.h>
int main()
{
    int a, *p;
    a = 10;
    p = &a;
    printf("%d \n", *p);
    printf("%p \n", &a);
    printf("%p \n", p);
    printf("%p \n", &p);
    return 0;
}
```

output

10

00000000000061FE1C

00000000000061FE1C

000000000000661FE10

Lab Program - 02

Q) Write a program to calculate the length of the string using a pointer

```
#include <stdio.h>
#include <conio.h>
int main()
{
    char s[100], *p;
    int i;
    printf("Enter a string: ");
    gets(s);
    p = s;
    while (*p != '\0')
    {
        i++;
        p++;
    }
    printf("Length of string is %d", i);
    return 0;
}
```

Output

Enter a string 'Neha'

Length of string is 4

Lab Program -03

③ Write a Program to swap two numbers using pointer

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{
```

```
int a, b, c, *p, *q;
```

```
printf("enter the numbers:");
```

```
scanf("%d %d", &a, &b);
```

```
p = &a;
```

```
q = &b;
```

```
printf("before swapping a=%d b=%d", *p, *q);
```

```
c = *p;
```

```
*p = *q;
```

```
*q = c;
```

```
printf("\n after swapping a=%d b=%d", *p, *q);
```

```
return 0;
```

```
}
```

Output

enter the numbers 10 20

before Swapping a=10 b=20

after swapping a=20 b=10

Lab Program - 04

④ Write a Program to print all permutations

```
#include <stdio.h>
```

```
#include <string.h>
```

```
void swap(char *ch1, char *ch2)
```

```
{
```

```
    char tmp;
```

```
    tmp = *ch1;
```

```
    *ch1 = *ch2;
```

```
    *ch2 = tmp;
```

```
}
```

```
void permute(char *a, int l, int r)
```

```
{
```

```
    if (l == r)
```

```
        printf("%s\n", a);
```

```
    else
```

```
    {
```

```
        for (int i = l; i <= r; i++)
```

```
        {
```

```
            swap(&a[l], &a[i]);
```

```
            permute(a, l+1, r);
```

```
            swap(&a[l], &a[i]);
```

```
        }
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    char str[40];
```

```
    printf("enter a string\n");
```

```
    scanf("%s", str);
```

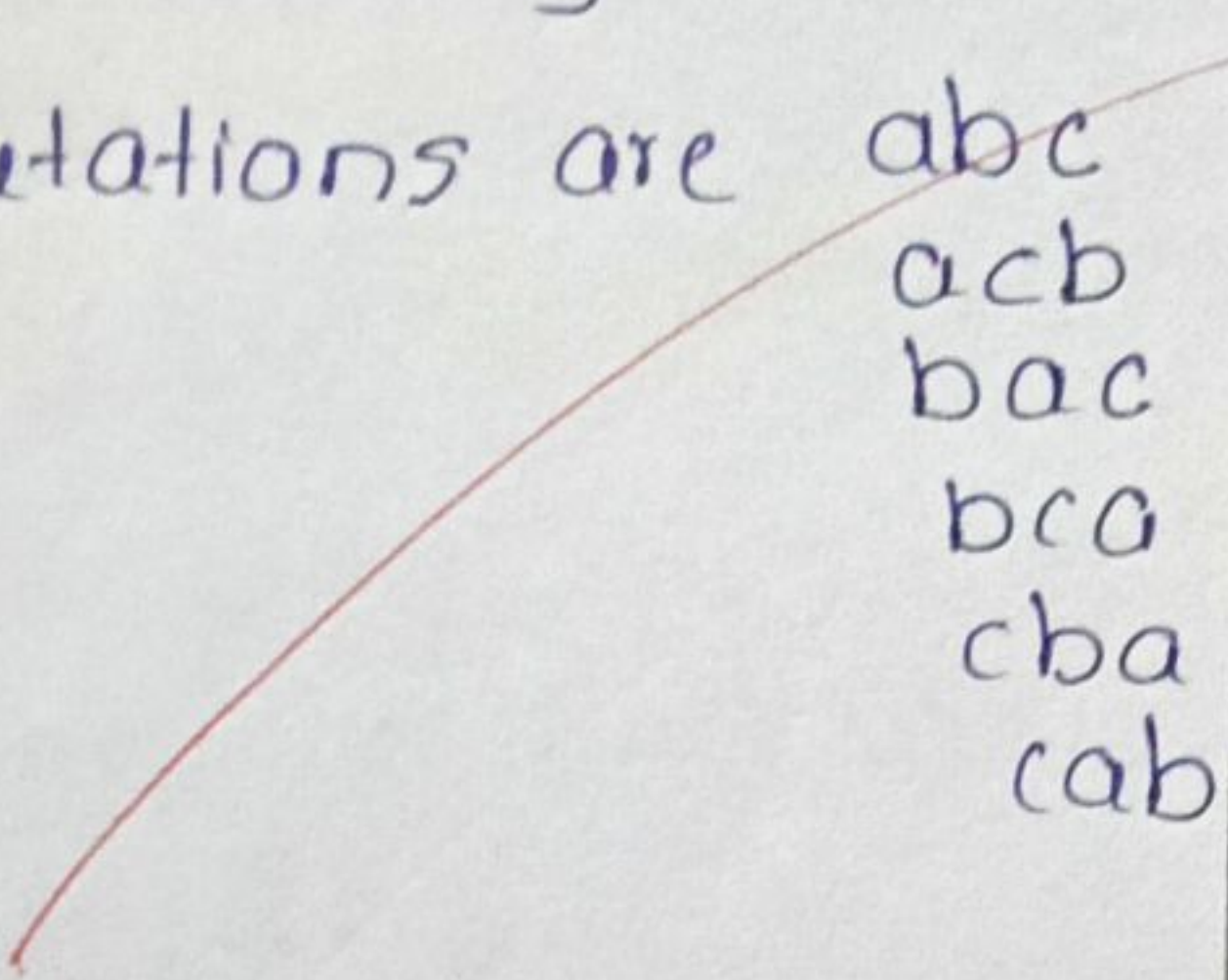


```
printf(" permutations are");  
permute(str, 0, n-1);  
return 0;  
}
```

Output

enter a string abc

Permutations are abc
acb
bac
bca
cba
cab



Lab Program - 05

⑤ Write a program to store n student information using structure

```
#include <stdio.h>
#include <conio.h>
struct student {
    char name[20];
    int rollnum;
    char address[75];
    float marks;
} S[10];

int main()
{
    int i, n;
    printf("enter the limit \n");
    scanf("%d", &n);
    for (i = 0; i < n; i++)
    {
        printf("enter the name \n");
        scanf("%s", &S[i].name);
        printf("enter the rollnum \n");
        scanf("%d", &S[i].rollnum);
        printf("enter the address \n");
        scanf("%s", &S[i].address);
        printf("enter marks \n");
        scanf("%f", &S[i].marks);
    }
```



```

printf("Student Information");
for(i=0; i<n; i++)
{
printf(" \n name is %s ", s[i].name);
printf(" \n rollnum is %d", s[i].rollnum);
printf(" \n marks is %f", s[i].marks);

}
return 0;
}

```

Output

```

enter the limit 2
enter the name manasa
enter the rollnum 77
enter the address katgal
enter marks 88
enter the name Manish
enter roll num 78
enter the address Katgal
enter marks 89
Student information
name is manasa
rollnum is 77
address is katgal
marks is 88.00
name is Manish
rollnum is 78
address is katgal
marks is 89

```


⑥ Write a program to implement push, pop & traverse operation on ~~sa~~ STACK.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#define size 5
```

```
void push();
```

```
void pop();
```

```
void display();
```

```
int stack[size], i, top = -1, ch, num;
```

```
void main()
```

```
{
```

```
printf("Stack implementation \n");
```

```
printf("1. push \n");
```

```
printf("2. pop \n");
```

```
printf("4. exit \n");
```

```
do
```

```
{
```

```
printf("\n enter the choice : ");
```

```
scanf("%d", &ch);
```

```
switch (ch)
```

```
{
```

```
case 1:
```

```
    push();
```

```
    break;
```

```
case 2:
```

```
    pop();
```

```
    break;
```


case 3:

display();

break;

default:

printf("Invalid choice");

}

}

while (ch <= 4);

getch();

}

void push()

{

if (top == size - 1)

{

printf("Stack is full");

}

else

{

printf("enter the number:");

scanf("%d", &num); top = top + 1;

stack[top] = num;

}

}

void pop()

{

if (top == -1)


```
{
    printf("stack is empty\n");
}
else
{
    printf("deleted elements is : %d \n",
           stack[top]);

    top = top - 1;
}
}
```

```
void display()
{
    if (top == -1)
    {
        printf("stack is empty no elements to
               display\n");
    }
    else
    {
        printf("The stack elements are : ");
        for (i = 0; i <= top; i++)

            printf("%d\t", stack[i]);
    }
}
```


Output

Stack implementation

1. Push
2. Pop
3. display
4. exit

enter a choice : 1
enter the number : 10
enter a choice : 1
enter the number : 20
enter a choice : 1
enter the number : 30
enter a choice : 1
enter the number : 40
enter the choice : 1
enter the number : 50
enter the choice : 1
Stack is full
enter choice 3
Stack elements are
10
20
30
40
50

enter choice : 2
deleted element is 50
enter choice : 2
deleted element is 40
enter choice : 2
deleted element is 30

Lab Program - 07

Q7) Write a program to convert infix notation to postfix notation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX_SIZE 100
```

```
typedef struct {
```

```
    char stack[MAX_SIZE];
```

```
    int top;
```

```
} stack;
```

```
void push(stack *s, char c)
```

```
{
```

```
    s->stack[++s->top] = c;
```

```
}
```

```
char pop(stack *s)
```

```
{
```

```
    return s->stack[s->top--];
```

```
}
```

```
int precedence(char c)
```

```
{
```

```
    return (c == '+' || c == '-') ? 1 : (c == '*' || c == '/') ?
```

```
    2 : 0;
```

```
}
```

```
void infixToPostfix(char *infix, char *postfix)
```

```
{
```

```
    stack operators = &stack, top = -1;
```

```
    int j = 0;
```

```
    for (int i = 0; infix[i]; i++)
```

```
    {
```

```
        if (isalnum(infix[i]))
```



```

postfix[j++] = infix[i];
else if (infix[i] == '(')
    push(&operators, infix[i]);
else if (infix[i] == ')') {
    while (operators.top != -1 &&
        operators.stack[operators.top] != '(') postfix[j++] =
        pop(&operators);
    pop(&operators);
}

```

```

else {
    while (operators.top != -1 &&
        precedence(operators.stack[operators.top]) >=
        precedence(infix[i]))
        postfix[j++] = pop(&operators);
    push(&operators, infix[i]);
}

```

```

while (operators.top != -1) postfix[j++] = pop(&operators);

```

```

postfix[j] = '\0';
}

```

```

int main()

```

```

{

```

```

    char infix[MAX_SIZE], postfix[MAX_SIZE];

```

```

    printf("Enter an infix expression:");

```

```

    fgets(infix, MAX_SIZE, stdin);

```

```

    infix[strcspn(infix, "\n")] = '\0';

```

```

    infixToPostfix(infix, postfix);

```


14

```
printf("Postfix expression: %s\n", postfix);  
return 0;  
}
```

Output.

Enter an infix expression $A+B*C$
postfix expression $ABC*+$

Lab Program - 08

Q8) Write a program to convert infix notation to prefix notation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define MAX_SIZE 100
```

```
typedef struct {
```

```
    char stack[MAX_SIZE];
```

```
    int top;
```

```
} stack;
```

```
void push(stack *s, char c)
```

```
{
```

```
    s->stack[++s->top] = c;
```

```
}
```

```
char pop(stack *s)
```

```
{
```

```
    return s->stack[s->top--];
```

```
}
```

```
int precedence (char c)
```

```
{
```

```
    return (c == '+' || c == '-') ? 1 : (c == '*' || c == '/') ?
```

```
    2 : 0;
```

```
}
```

```
void infixToPrefix(char *infix, char *prefix)
```

```
{
```

```
    stack operators = {, top = -1};
```

```
    int len = strlen(infix);
```

```
    char reversed[len+1];
```

```
    for (int i = 0; i < len; i++) reversed[i] = infix[len-i-1];
```


10
if (reversed[i] == '(')

reversed[i] = ')';

else if (reversed[i] == ')')

reversed[i] = '(';

}

reversed[len] = '\0';

for (int i = 0; j = 0; i < len; i++)

{

if (isalnum(reversed[i]))

prefix[j++] = reversed[i];

else if (reversed[i] == '(')

push(&operators, reversed[i]);

else if (reversed[i] == ')')

{

while (operators.top != -1 &&

operators.stack[operators.top] != '(')

prefix[j++] = pop(&operators);

pop(&operators);

}

else

{

while (operators.top != -1 &&

precedence(operators.stack[operators.top]) >=

precedence(reversed[i]))

prefix[j++] = pop(&operators);

push(&operators, reversed[i]);

}

}

while (operators.top != -1)

prefix[strlen(prefix)] = pop(&operators);

prefix[strlen(prefix)] = '\0';


```

len = strlen(prefix);
for (int i = 0; i < len / 2; i++)
{
    char temp = prefix[i];
    prefix[i] = prefix[len - i - 1];
    prefix[len - i - 1] = temp;
}
}

```

```

int main()

```

```

{
    char infix[MAX_SIZE], prefix[MAX_SIZE];
    printf("Enter an infix expression; ");
    fgets(infix, MAX_SIZE, stdin);
    infix[strlen(infix, "\n")] = '\0';
    infixToPrefix(infix, prefix);
    printf("Prefix expression: %s\n", prefix);
    return 0;
}

```

Output

Enter an infix expression A*B

Prefix expression *AB

Lab Program - 09

Q9) Write a program to convert prefix notation to postfix notation

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#define MAX_SIZE 100
```

```
typedef struct
```

```
{  
    char items[MAX_SIZE];
```

```
    int top;
```

```
} stack;
```

```
void push(stack *s, char c)
```

```
{  
    s->items[++s->top] = c;
```

```
}
```

```
char pop(stack *s)
```

```
{  
    return s->items[s->top--];
```

```
}
```

```
int isOperator(char c)
```

```
{  
    return c == '+' || c == '-' || c == '*' || c ==  
           '/';
```

```
}
```

```
void prefixToPostfix(char *prefix, char *postfix)
```

```
{
```

```
    stack s = {.top = -1};
```

```
    int len = strlen(prefix);
```

```
    for(int i = len - 1; i >= 0; i--) {  
        if (isOperator(prefix[i]))
```



```

    {
        postfix[strlen(postfix)] = pop(&s);
        postfix[strlen(postfix)] = pop(&s);
        postfix[strlen(postfix)] = prefix[i];
        push(&s, postfix[strlen(postfix)-1]);
        push(&s, postfix[strlen(postfix)-2]);
    }
else
    {
        push(&s, prefix[i]);
    }
}
}

int main()
{
    char prefix[MAX_SIZE], postfix[MAX_SIZE];
    printf("Enter prefix expression:");
    fgets(prefix, MAX_SIZE, stdin);
    prefixToPostfix(prefix, postfix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}

```

Output

Enter prefix expression	*AB
Postfix expression	AB*

Lab Program - 10

10) Write a program to perform the operation Insert, and Display on Queue.

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
void insert();
```

```
void del();
```

```
void display();
```

```
int front = -1, rear = -1;
```

```
int queue[5];
```

```
void main()
```

```
{
```

```
int ch;
```

```
while (ch != 4)
```

```
{
```

```
printf("\n***** Main Menu*****");
```

```
printf("\n 1. Insert \n 2. Delete \n 3. Display \n
```

```
4. Exit \n");
```

```
printf("\n Enter your choice");
```

```
scanf("%d", &ch);
```

```
{
```

```
case 1:
```

```
insert();
```

```
break;
```

```
case 2:
```

```
del();
```

```
break;
```


case 3:

display();

break;

case 4:

exit(0);

break;

default:

printf("\n enter valid choice \n");

}

}

getch();

}

void insert()

{

int item;

printf("\n enter the element \n");

scanf("%d", &item);

if (rear == 4)

{

printf("\n OVERFLOW \n");

return;

}

if (front == -1 && rear == -1)

{

front = 0;

rear = 0;

}

else

{

rear = rear + 1;

}

22
queue[rear] = item;

printf("\n value inserted");

}

void del()

{

int item;

if (front == -1 || front > rear)

{
printf("\n UNDERFLOW \n"); return;

}

else {

item = queue[front]; if (front == rear) {

{

front = -1; rear = -1;

}

else {

front = front + 1;

}

printf("\n value deleted");

}

}

void display()

{

int i;

if (rear == -1)

{

printf("\n empty queue \n");

}

else

{

printf("\n queue values \n");


```

for (i = front; i < rear; i++)
{
    printf("%d\n", queue[i]);
}
}

```

output

Main Menu

1. Insert
2. Delete
3. Display
4. exit

enter your choice : 1

enter the element : 10

enter your choice : 1

enter the element : 20

enter your choice : 1

enter the element : 30

enter your choice : 1

enter the element : 40

enter your choice : 1

enter the element : 50

enter your choice : 1

OVERFLOW

enter your choice 3

Queue values

10
20
30
40
50

enter choice 2

value deleted 10

Lab Program -11

⑪ Write program to implement circular queue.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
#define max 6
```

```
int queue[max], front = -1, rear = -1;
```

```
void enqueue(int element)
```

```
{
```

```
    if (front == -1 && rear == -1)
```

```
    {
```

```
        front = 0; rear = 0;
```

```
        queue[rear] = element;
```

```
    }
```

```
else if ((rear+1)%max == front)
```

```
    {
```

```
        printf("Queue is overflow..");
```

```
    }
```

```
else
```

```
{
```

```
    rear = (rear+1)%max;
```

```
    queue[rear] = element;
```

```
}
```

```
}
```

```
int dequeue()
```

```
{
```

```
    if (front == -1 && rear == -1)
```

```
    {
```

```
        printf("\nQueue is underflow..");
```

```
    }
```

```
else if (front == rear)
```



```

{
    printf("\n The dequeued element is %d", queue
                                                [front]);
    front = -1;
    rear = -1;
}
else
{
    printf("\n The dequeued element is %d", queue
                                                [front]);
    front = (front + 1) % max;
}
}

```

```

void display()
{
    int i = front;
    if (front == -1 && rear == -1)
    {
        printf("\n Queue is empty...");
    }
    else
    {
        printf("\n Elements in a Queue are:");
        while (i <= rear)
        {
            printf("%d, ", queue[i]);
            i = (i + 1) % max;
        }
    }
}

```

```

void main()

```

```

{
    int choice = 1, 2;
    while (choice < 4 && choice != 0)

```



```

{
printf("\n 1: Insert an element");
printf("\n 2: Delete an element");
printf("\n 3: Display the element");
printf("\n Enter your choice");
scanf("%d", &choice);
switch(choice)
{
case 1:
printf("Enter the element which is to be inserted");
scanf("%d", &x);
enqueue(x);
break;
case 2:
dequeue();
break;
case 3:
display();
}
getch();
}

```

Output

1: Insert an element
2: Delete an element
3: Display the element
Enter your choice 1.

Enter the element which is to be inserted 10

Enter your choice 1.

Enter the element which is to be inserted 20

enter your choice 1

Enter the element which is to be inserted 30

enter your choice 3

Elements in a Queue are: 10, 20, 30

Enter your choice 2

The dequeued element is 20 10

Enter your choice 2

The dequeued element is 20

Enter your choice 2

The dequeued element is 30

Enter your choice 2

Queue is underflow....

Lab Program -12

⑫ Write program to implement double ended queue.

```
#include <stdio.h>
```

```
#define size 5
```

```
int dequeue[size];
```

```
int f = -1, r = -1;
```

```
void insert-front(int x)
```

```
{
```

```
if ((f == 0 && r == size - 1) || (f == r + 1))
```

```
{
```

```
printf("Overflow");
```

```
}
```

```
else if ((f == -1) && (r == -1))
```

```
{
```

```
f = r = 0;
```

```
dequeue[f] = x;
```

```
}
```

```
else if (f == 0)
```

```
{
```

```
f = size - 1;
```

```
dequeue[f] = x;
```

```
}
```

```
else
```

```
{
```

```
f = f - 1;
```

```
dequeue[f] = x;
```

```
}
```

```
}
```



```
void insert - rear (int x)
```

```
{  
    if ((f == 0 && r == size - 1) || (f == r + 1))
```

```
{  
    printf("Overflow");
```

```
}  
else if ((f == -1) && (r == -1))
```

```
{  
    r = 0;  
    deque[r] = x;
```

```
}
```

```
}
```

```
void display()
```

```
{  
    int i = f;
```

```
    printf("\n Elements in a deque are:");
```

```
    while (f != r)
```

```
{  
        printf("%d ", deque[i]);  
        i = (i + 1) % size;
```

```
}
```

```
    printf("%d ", deque[r]);
```

```
}
```

```
void get front()
```

```
{  
    if ((f == -1) && (r == -1))
```

```
{  
        printf("Deque is empty");
```

```
}
```

```
else
```

```
{  
    printf("\n The value of the element at  
    front is: %d", deque[f]);
```



```
}  
}  
void getrear()
```

```
{  
    if ((l == -1) && (r == -1))
```

```
{  
        printf("Deque is empty");  
    }
```

```
else  
{  
    printf("\n The value of the element at rear is  
    %d", deque[r]);  
}
```

```
}  
void delete-front()
```

```
{  
    if ((l == -1) && (r == -1))
```

```
{  
        printf("Deque is empty");  
    }
```

```
else if (l == r)
```

```
{  
    printf("\n The deleted element is %d", deque[l]);
```

```
    l = -1;
```

```
    r = -1;
```

```
}
```

```
else if (l == (size - 1))
```

```
{  
    printf("\n The deleted element is %d",  
    deque[l]);
```

```
    l = 0;
```

```
}
```


else

{

printf("\n The deleted element is %d", deque[l]);

l = l + 1;

}

}

void delete_rear()

{

if ((l == -1) && (r == -1))

{

printf("Deque is empty");

}

else if (l == r)

{

printf("\n The deleted element is %d",
deque[r]); l = -1;

r = -1;

}

else if (r == 0)

{

printf("\n The deleted element is %d", deque[r]);

r = size - 1;

}

else

{

printf("\n The deleted element is %d", deque[r]);

r = r - 1;

}

}

int main()

{


```

insert - front(20);
insert - front(10);
insert - rear(30);
insert - rear(50);
insert - rear(80);
display(); //calling the display function to
            retrieve the values of deque
values of deque getfront(); getfront(); // Retrieve the value
            at front - end
getrear(); // Retrieve the value at rear end
delete - front();
delete - rear();
display();
// calling display function to retrieve values after
deletion
return 0;
}

```

Output

1. Insert an element
2. Delete an element
3. Display the element.

Enter your choice

Enter the element which is to be inserted, 10 20 30 50 80.

Elements in deque are : 10 20 30 50 80

Front element : 10

Rear element : 80

Deleted^{front} element : 10

Deleted rear element : 80

Elements in deque are : 20 30 50

Lab Program -13

⑬ Write program to implement priority queue.

```
#include <stdio.h>
#include <stdlib.h>
int heap[40];
int size = -1;

int parent(int i)
{
    return (i-1) / 2;
}

int left-child(int i)
{
    return i+1;
}

int right-child(int i)
{
    return i+2;
}

int get-Max()
{
    return heap[0];
}

int get-Min()
{
    return heap[size];
}

void moveUp(int i)
{
    while (i > 0)
```



```
{ if (heap[parent(i)] < heap[i])
```

```
{
```

```
    int temp;
```

```
    temp = heap[parent(i)];
```

```
    heap[parent(i)] = heap[i];
```

```
    heap[i] = temp;
```

```
}
```

```
i = i/2;
```

```
}
```

```
}
```

```
void moveDown(int k)
```

```
{
```

```
    int index = k;
```

```
    int left = left-child(k);
```

```
    if (left <= size && heap[left] > heap[index])
```

```
    {
```

```
        index = left;
```

```
    }
```

```
    int right = right-child(k);
```

```
    if (right <= size && heap[right] > heap[index])
```

```
    {
```

```
        index = right;
```

```
    }
```

```
    if (k != index)
```

```
    {
```

```
        int temp;
```

```
        temp = heap[index];
```

```
        heap[index] = heap[k];
```

```
        heap[k] = temp;
```


moveDown(indx);

}

}

void removeMax()

{

int r = heap[0];

heap[0] = heap[size];

size = size - 1;

moveDown(0);

}

void insert(int p)

{

size = size + 1;

heap[size] = p;

moveUp(size);

}

void delete(int i)

{

heap[i] = heap[0] + 1

moveUp(i);

removeMax();

}

int main()

{

insert(20);

insert(19);

insert(21);

insert(18);

insert(12);

insert(17);


```
insert(15);
```

```
insert(16);
```

```
insert(14);
```

```
int i=0;
```

```
printf("Elements in a priority queue are: ");
```

```
for (int i=0; i<size; i++)
```

```
{  
    printf("%d", heap[i]);
```

```
}
```

```
delete(2);
```

```
printf("\nElements in a priority queue after  
deleting the elements are: ");
```

```
for (int i=0; i<size; i++)
```

```
{
```

```
    printf("%d", heap[i]);
```

```
}
```

```
int max = get_Max();
```

```
printf("\n The element which is having the  
highest priority is %d", max);
```

```
int min = get_Min();
```

```
printf("\n The element which is having the  
minimum priority is: %d", min);
```

```
return 0;
```

```
}
```


Output

Elements in a priority queue are 21 20 19 16 12 17
15 18 14

Elements in a priority queue after deleting the
element are : 21 20 17 16 12 14 15 18

The element which is having the highest priority
is 21

The element which is having the minimum
priority is 12

Lab Program - 14

- ⑭ Write a program to search an element using linear search

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n, i, a[10], x, found = 0;
    printf("enter the limit");
    scanf("%d", &n);
    printf("enter %d elements", n);
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);
    printf("enter the element to be searched");
    scanf("%d", &x);
    for (i = 0; i < n; i++)
    {
        if (x == a[i])
        {
            found = 1;
            break;
        }
    }
    if (found == 1)
        printf("element found at %d position", i);
}
```



```
else  
    printf("Element is not found");
```

```
getch();
```

```
}
```

Output

enter a limit 4

enter 4 elements 10

20

30

40

enter the element to be searched 30

Element found at 2 position

Lab Program - 15

⑤ Program: Write a program to sort given Array using Insertion sort technique

```
#include <stdio.h>
```

```
void insert();
```

```
int a[20], n, i, j, temp;
```

```
void main()
```

```
{
```

```
printf("Enter the limit:");
```

```
scanf("%d", &n);
```

```
printf("Enter the %d elements: ", n);
```

```
for (i = 0; i < n; i++)
```

```
scanf("%d", &a[i]);
```

```
printf("Before sorting array elements are - \n");
```

```
for (i = 0; i < n; i++)
```

```
printf("%d", a[i]);
```

```
insert(a, n);
```

```
printf("\n After sorting array elements are - \n");
```

```
for (i = 0; i < n; i++)
```

```
printf("%d", a[i]);
```

```
getch();
```

```
}
```

```
void insert()
```

```
{
```

```
for (i = 1; i < n; i++)
```

```
{
```

```
temp = a[i];
```



```

j = i-1;
while (j >= 0 && temp < a[j])
{
    a[j+1] = a[j];
    j = j-1;
}
a[j+1] = temp;
}
}

```

Output

Enter the limit 4.

Enter 4 elements 100
50
~~500~~
2

Before sorting array elements are

After sorting array elements are

2
50
100
500

Lab Program - 16.

①6) Write a program to sort given Array using Bubble sort technique.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
int a[50], n, i, j, temp = 0;
```

```
printf("enter the limit for the array:");
```

```
scanf("%d", &n);
```

```
printf("enter the %d elements for the array\n", n);
```

```
for(i = 0; i < n; i++)
```

```
{
```

```
scanf("%d", &a[i]);
```

```
}
```

```
for(i = 0; i < n - 1; i++)
```

```
{
```

```
for(j = 0; j < n - i - 1; j++)
```

```
{
```

```
if(a[j] > a[j+1])
```

```
{
```

```
temp = a[j];
```

```
a[j] = a[j+1];
```

```
a[j+1] = temp;
```

```
}
```

```
}
```

```
}
```



```
printf ("Sorted elements are: ");
```

```
for (i = 0; i < n; i++)
```

```
printf ("%d ", a[i]);
```

```
getch();
```

```
}
```

Output

enter the limit for the array 4

enter 4 elements for the array 200

40

80

100

Sorted elements are

40

80

100

200

Lab Program -17

- (17) Write a program to sort given Array using Quick sort technique.

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
int i, n, a[25];
```

```
printf("Enter the limit: ");
```

```
scanf("%d", &n)
```

```
printf("Enter %d elements: ", n);
```

```
for(i=0; i<n; i++)
```

```
scanf("%d", &a[i]);
```

```
quicksort(a, 0, n-1);
```

```
printf("The sorted order is: ");
```

```
for(i=0; i<n; i++)
```

```
printf("%d", a[i]);
```

```
getch();
```

```
}
```

```
void quicksort(int a[25], int first, int last)
```

```
{
```

```
int i, j, pivot, temp;
```

```
if (first < last)
```

```
{
```

```
    pivot = first;
```

```
    i = first;
```

```
    j = last;
```

```
    while (i < j)
```

```
    {
```

```
        while (a[i] <= a[pivot] && a[j] < a[pivot])
```

```
            i++;
```


while (a[j] > a[pivot])

j--;

if (i < j)

{

temp = a[i];

a[i] = a[j];

a[j] = temp;

}

temp = a[pivot];

a[pivot] = a[j];

a[j] = temp;

quicksort(a, first, j-1)

quicksort(a, j+1, last);

}

}

Output

Enter the limit 4

Enter 4 elements 50

600

90

5

The sorted order is 5

50

90

600

Lab Program - 18

- (18) Write a program to sort given Array using selection sort technique.

```
#include <stdio.h>
#include <conio.h>
void selection();
int a[20], n, i, j, small, temp;
void main()
{
    printf("Enter the limit");
    scanf("%d", &n);
    printf("Enter the %d elements: ", n);
    for (i=0; i<n; i++)
        scanf("%d", &a[i]);
    printf("Before sorting array elements are -\n");
    for (i=0; i<n; i++)
        printf("%d", a[i]);
    selection(a, n);
    printf("\nAfter sorting array elements are -\n");
    for (i=0; i<n; i++)
        printf("%d", a[i]);
    getch();
}

void selection()
{
    for (i=0; i<n-1; i++)
    {
        small = i;
        for (j=i+1; j<n; j++)
            if (a[j] < a[small])
                small = j;
    }
}
```



```
    tmp = a[smallest];  
    a[smallest] = a[i];  
    a[i] = tmp;  
}  
}
```

Output

Enter the limit 4

Enter 4 elements 100

20

500

6

Before sorting array elements are

100

20

500

6

After sorting array elements are

6

20

100

500

(19) Write a program to implement singly linked list

```
#include <stdio.h>
#include <stdlib.h>
struct node
{
    int data;
    struct node *next;
};
struct node *head;
void begininsert();
void begin-delete();
void display();
void search();
void main()
{
    int choice = 0;
    while (choice != 5)
    {
        printf("\n\n***** Main Menu *****\n");
        printf("\n1. Insert in beginning\n2. Delete from\n\nBeginning\n3. Search for an element\n4. Show\n\n5. Exit\n");
        printf("\nEnter your choice ? \n"); scanf("%d", &choice);

        switch(choice)
        {
            case 1 :
                begininsert();
                break;
```


case 2:

begin-delete();

break;

case 3:

search();

break;

case 4:

display();

break;

case 5:

exit(0);

break;

default:

printf("Please enter valid choice ...");

}

}

}

void begininsert()

{

struct node * ptr;

int item;

ptr = (struct node*) malloc(sizeof(struct node));

if (ptr == NULL)

{

printf("\n OVERFLOW");

}

else

{

printf("\nEnter value\n");


```

scanf("%d", &item);
ptr->data = item;
ptr->next = head;
head = ptr;
printf("\nNode inserted");
}

```

```

}
void begin_delete()
{
    struct node *ptr;
    if (head == NULL)
    {
        printf("\nList is empty\n");
    }
    else
    {
        ptr = head;
        head = ptr->next;
        free(ptr);
        printf("\nNode deleted from the beginning...\n");
    }
}

```

```

void search()
{
    struct node *ptr;
    int item, i=0, flag;
    ptr = head;
    if (ptr == NULL)
    {
        printf("\nempty list\n");
    }
}

```


else

{

printf("\n Enter item which you want to search?\n");

scanf("%d", &item);

while (ptr != NULL)

{

if (ptr->data == item)

{

printf("item found at location %d", i+1);

}

else

{

i++;

flag = 0;

}

ptr = ptr->next;

}

if (flag == 1)

{

printf("Item not found\n");

}

}

}

void display()

{

struct node *ptr; ptr = head;

if (ptr == NULL)

{

printf("Nothing to print");

}

else


```

{
    printf("\n printing values \n");
    while (ptr != NULL)
    {
        printf("\n %d", ptr->data);
        ptr = ptr->next;
    }
}

```

Output

*** Main Menu ***

1. Insert at beginning
2. Delete from beginning
3. Search for an element
4. Show
5. exit

enter your choice: 1

Enter value : ~~20~~ 10

Node inserted

enter your choice: 1

Enter value : 20

Node inserted

Enter your choice: 1

Enter value : 30

Node inserted

Enter your choice: 4

Printing values:

30 -> 20 -> 10 -> NULL

Enter your choice: 3

Enter item to search: 20

Item found at location: 3

Enter your choice 2

Node deleted from the beginning

Enter your choice:

Printing value:

30 -> 20 -> 10 -> NULL

enter your choice

// program ends

Lab Program -20

Q20) Write program to implement double linked list

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct node
```

```
{
```

```
    struct node *prev;
```

```
    struct node *next;
```

```
    int data;
```

```
};
```

```
struct node *head;
```

```
void insertion -beginning();
```

```
void deletion -beginning();
```

```
void display();
```

```
void search();
```

```
void main()
```

```
{
```

```
    int choice = 0;
```

```
    while (choice != 4)
```

```
{
```

```
    printf("\n choose one option from the following  
list --- \n");
```

```
    printf("\n 1. Insert in beginning\n 2. Delete from  
Beginning\n 3. Search\n 4. Show\n 5. Exit \n");
```

```
    printf("\n enter your choice ? \n");
```

```
    scanf("\n %d", &choice);
```

```
    switch (choice)
```

```
{
```

```
        case 1:
```

```
            insertion -beginning();
```


55
break;

case 2:

deletion - beginning();

case 3:

search();

break;

case 4:

display();

break;

case 5:

exit(0);

break;

default:

printf("please enter valid choice ...");

}

}

}

void insertion - beginning()

{

struct node *ptr; int item;

ptr = (struct node *) malloc (sizeof (struct node));

if (ptr == NULL)

{

printf("\n OVERFLOW");

}

else

{

printf("\n enter item value");

scanf("%d", &item);


```
if (head == NULL)
```

```
{
```

```
ptr -> next = NULL;
```

```
ptr -> prev = NULL;
```

```
ptr -> data = item;
```

```
head = ptr;
```

```
}
```

```
else
```

```
{
```

```
ptr -> data = item;
```

```
ptr -> prev = NULL;
```

```
ptr -> next = head;
```

```
head -> prev = ptr;
```

```
head = ptr;
```

```
}
```

```
printf("\n Node inserted\n");
```

```
}
```

```
}
```

```
void deletion-beginning()
```

```
{
```

```
struct node *ptr; if (head == NULL)
```

```
{
```

```
printf("\n UNDERFLOW");
```

```
}
```

```
else if (head -> next == NULL)
```

```
{
```

```
head = NULL; free(head);
```



```
printf("\n node deleted\n");
```

```
}
```

```
eye
```

```
{
```

```
ptr = head;
```

```
head = head->next;
```

```
head->prev = NULL;
```

```
free(ptr);
```

```
printf("\n node deleted\n");
```

```
}
```

```
}
```

```
void display()
```

```
{
```

```
struct node *ptr;
```

```
printf("\n Printing values....\n");
```

```
ptr = head;
```

```
while (ptr != NULL)
```

```
{
```

```
printf("%d\n", ptr->data);
```

```
ptr = ptr->next;
```

```
}
```

```
}
```

```
void search()
```

```
{
```

```
struct node *ptr;
```

```
int item, i = 0, flag;
```

```
ptr = head;
```

```
if (ptr == NULL)
```



```

{
    printf("\n Empty list \n");
}
else
{
    printf("\n enter item which you want to search\n");
    scanf("%d", &item);
    while (ptr != NULL)
    {
        if (ptr->data == item)
        {
            i++;
        }
        else
        {
            printf("\n item found at location %d", i+1);
            flag = 0;
            break;
        }
        flag = 1;
        ptr = ptr->next;
    }
    if (flag == 1)
    {
        printf("\n Item not found \n");
    }
}
}

```


Lab Program 20

Output

Choose one option from the following list....

1. Insert at beginning
2. Delete from beginning
3. Search
4. Show
5. Exit

Enter your choice : 1

Enter item value : 10

Node inserted

Enter your choice : 1

enter item value : 20

Node inserted

Choose one option from the following list....

Enter your choice : 1

enter item value : 30

Enter your choice 4

Printing values : 30

20

10

Enter your choice : 3

enter item to search :

Item found at position 3

Enter your choice : 2

Node deleted

Enter your choice : 4

Printing values 20
10

Seen
9/6/25